VARSIVO

POE

Emihle Zifo - ST10452317

Bokgabo Mohlala - ST10440136

Mampe Shaan Motsholane - ST10437865

Keletso Hope Thamaga - ST10456948

Table of Contents

1. Introduction and Purpose of the App 4

1.1 The problem ..... Error! Bookmark not defined.

1.2 Our Solution ..... Error! Bookmark not defined.

2. Design Considerations ..... Error! Bookmark not defined.

2.1 Visual identity 4

2.2 User interface and navigation 5

2.3 Handling invalid input 5

2.4 Accessibility and data friendliness 5

2.5 Security 6

3.System architecture and technology 6

3.1 Source Code Repository 6

3.2 Why of database? 6

3.3 Firestore data model 6

4.REST API 6

5.Features 7

5.1 Registration and login 7

5.2 Single sign-on (Google) 7

5.3 Settings 8

5.4 Core Features 8

5. 5 NoSQL database (Firestore) 10

5. 6 Blob storage 10

5. 7 Offline mode with synchronization 10

5. 8 Real-time push notifications 10

5. 9 Multi-language support 10

6. App icon and final assets 10

7. GitHub Actions and automated testing 10

7. 1 The workflows 10

7. 2 What is tested 11

7. 2 Manual testing on a real phone 11

8. Preparing for the Google Play Store 11

8.1 Signed release build 11

8.2 Store listing 12

8.3 Play Console evidence 12

8.4. Pre-publication checklist 12

9. Release notes (Version 2.0) 12

9.1. Innovative features added 13

9.2. Improvements and fixes 13

10. Demonstration Video 13

📽 **Link to demo video:** `[PASTE UNLISTED YOUTUBE LINK HERE]` 13

11. Use of AI tools 14

12. References..... 15

PART 3: FINAL PORTFOLIO OF EVIDENCE

1. Introduction and Purpose of the App

1.1 The Problem

Navigating the transition from secondary to tertiary education in South Africa is notoriously complex. Grade 12 learners are forced to manage university applications, track varying bursary deadlines, calculate their Admission Point Score (APS), and understand specific subject prerequisites across dozens of unconnected websites. This fragmented landscape frequently results in missed funding opportunities and missed application windows, disproportionately impacting learners who lack access to private career counselling.

1.2 Our Solution

Varsivo was developed to consolidate this entire journey into one streamlined mobile application. By integrating university and bursary discovery, an accurate APS calculator, and application tracking into a single platform, Varsivo eliminates the guesswork. Furthermore, the app extends its support beyond matriculation through "UniAssist," a premium feature tier designed to assist first-year students with their academic timetables and deadlines.

1.3 Target Users

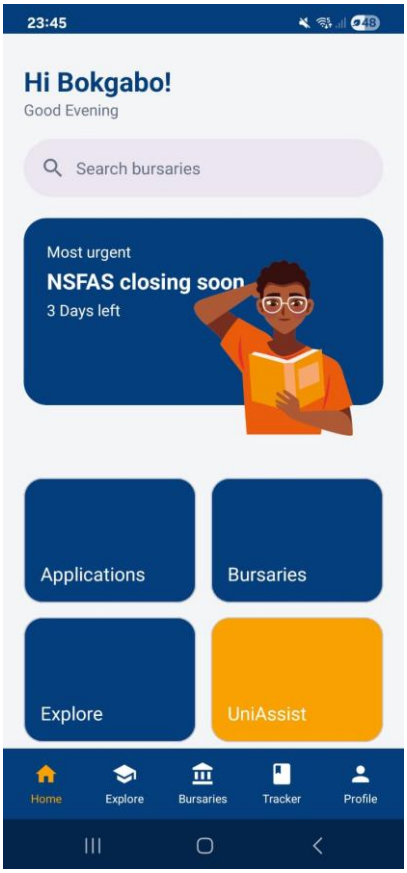
The primary audience consists of South African learners in Grades 11 and 12. A secondary audience includes first-year tertiary students who gain access to the UniAssist tier by verifying their institutional email addresses.

2.Design Considerations

The design and architecture of Varsivo were carefully considered to ensure a smooth user experience, robust security and scalability.

2.1 Visual identity

Varsivo employs a bespoke visual identity anchored by a navy blue (`#053E7D`) and gold (`#F8A100`) colour palette, selected to evoke a sense of academic ambition and prestige. The interface is constructed using Material 3 components, ensuring consistency across elements such as text inputs and informational cards.



2.2 User interface and navigation

To strictly comply with the project's Part 3 requirements, the application's user interface was completely migrated to **XML layouts and Android Views, eliminating all Jetpack Compose dependencies**. The architecture relies on a single `MainActivity` that hosts a `NavHostFragment`. A bottom navigation bar switches between five screens: Home, Explore, Bursaries, Tracker and Profile.

2.3 Handling invalid input

To ensure a robust user experience, all forms utilise pure validation functions (`Validators`) to catch invalid data before network requests are initiated. If a user encounters network instability, the app avoids crashing by displaying non-intrusive Snackbar messages equipped with a "Try Again" action.

2.4 Accessibility and data friendliness

The application prioritises accessibility by utilising standard Android UI components, which are inherently compatible with screen readers like TalkBack. Additionally, Varsivo is built offline-first. Critical functionalities, most notably the Application Tracker, remain fully operational without an active data connection

2.5 Security

Passwords are never stored in plain text. Firebase Authentication stores account passwords as salted scrypt hashes. For offline login, the phone keeps only a PBKDF2WithHmacSHA256 hash (120,000 iterations, 16-byte random salt), compared in constant time with `MessageDigest.isEqual`. Furthermore, sensitive assets like API keys and keystores are strictly excluded from version control via `.gitignore`.

3.System architecture and technology

3.1 Source Code Repository

The complete, version-controlled source code for the final submission is publicly hosted on GitHub at: **https://github.com/BokgaboM/Varsivo-OPSC**

3.2 Why of database?

Handling user authentication, synchronising application data across multiple devices, and serving dynamic content required a robust cloud database. A NoSQL document database, specifically Firebase (Firestore), was selected. This choice was driven by Firestore's real-time syncing capabilities, its seamless integration with the Android SDK, and its accessible free tier.

3.3 Firestore data model

The database is organised to store user data efficiently:

- `users/{uid}`: Stores email, name, school, role, language, notificationsEnabled, uniAssistVerified, photoBlob, fcmToken.
- `users/{uid}/applications/{id}`: Stores the institution, programme, type, and status of each tracked application.
- `bursaries/{id}`: Stores the name, provider, description, eligibility, field, and province of available bursaries.

4.REST API

Varsivo connects to external REST APIs through Retrofit: the Hipo Labs Universities API supplies the list of South African universities, and OpenStreetMap Nominatim converts a university name into map coordinates for the MapLibre map. Profile photos are uploaded and downloaded with direct HTTP calls to the Azure Blob Storage REST API. Bursaries are stored in and read from Cloud Firestore.

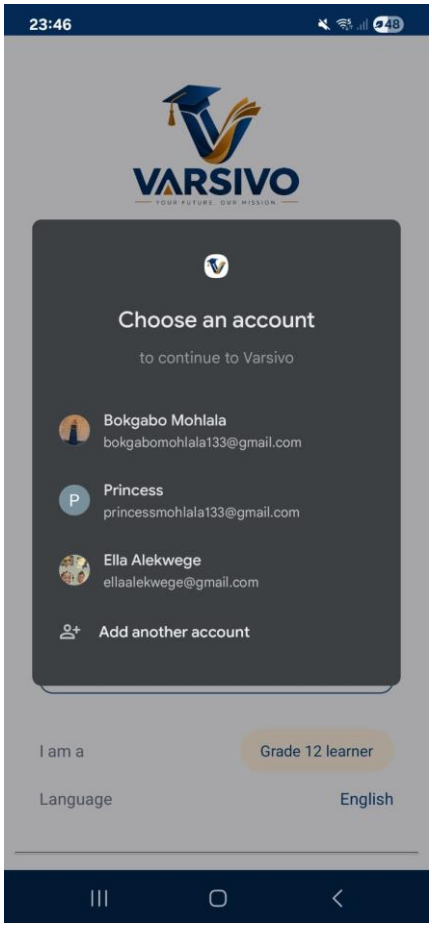
5.Features

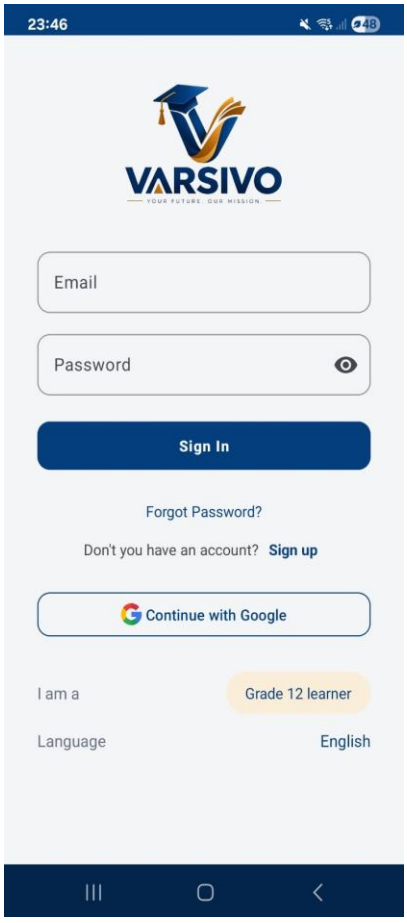
5.1 Registration and login

Users register with their email address and a password, or sign in with Google. User accounts are securely authenticated and managed via Firebase Authentication.

5.2 Single sign-on (Google)

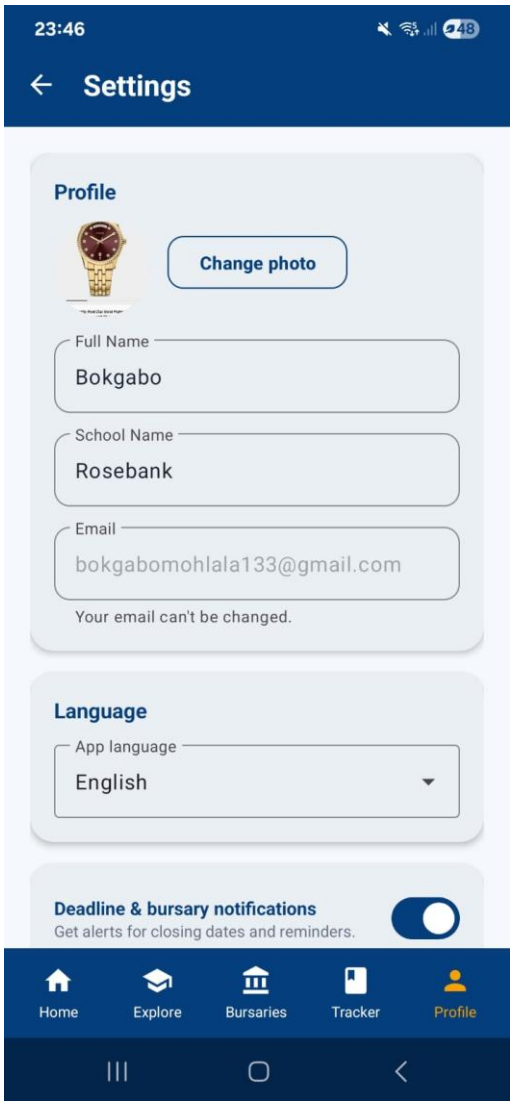
To reduce friction during onboarding, Google Sign-In has been integrated. This allows users to authenticate swiftly using their existing Google accounts without compromising security.





5.3 Settings

The settings menu allows users to update their profile information (name and school), change their app language, toggle push notifications, update their password, and securely log out.

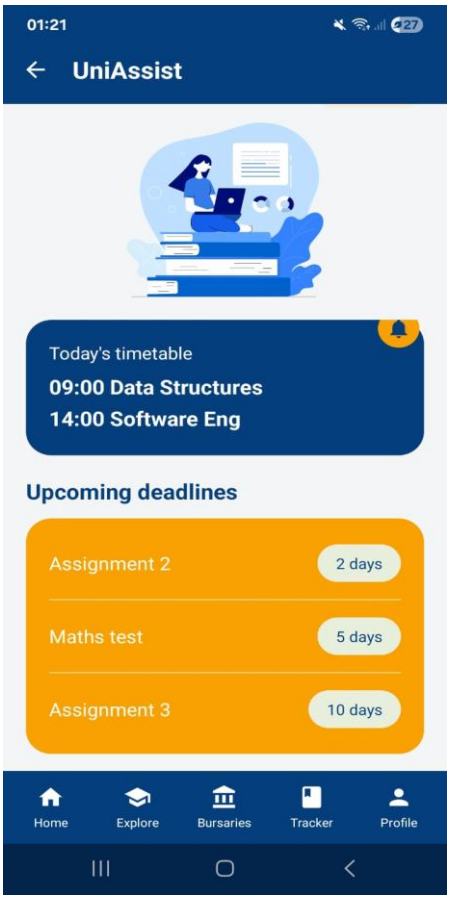


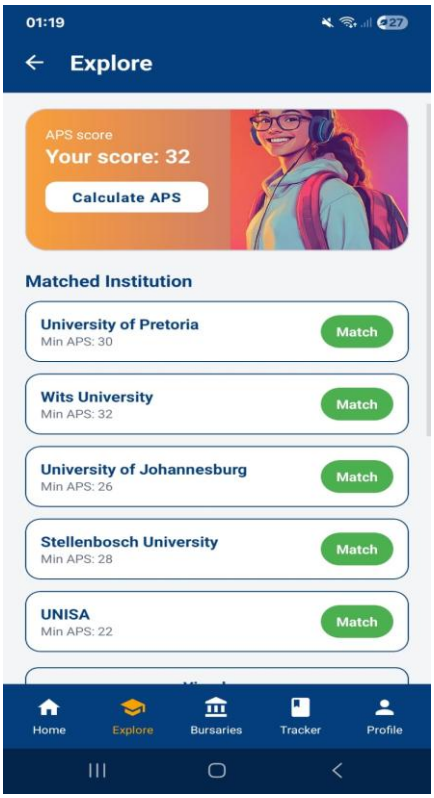
5.4 Core Features

- **APS Calculator:** Validates each mark (0–100), converts it to the 7-point APS scale, scales the score to six subjects and shows which institutions the learner qualifies for.
- **Popular Careers:** Shows a list of popular career paths for learners to explore.
- **Bursary Database:** Allows users to search, filter, and browse bursary listings from Firestore.
- **Application Tracker:** Empowers users to monitor the real-time status of their university and bursary applications.

- **UniAssist:** A premium tier that unlocks academic timetables and deadline tracking upon verification of a student email address.







5. 5 NoSQL database (Firestore)

Firestore is utilised to persist user settings and application data in real-time. Any changes made locally are instantly synchronised to the cloud, ensuring data integrity across devices.

5. 6 Blob storage

Users have the ability to upload a profile picture. These images are securely stored in **Azure Blob Storage**. The resulting download URL is saved to the user's Firestore document, allowing the image to be retrieved and displayed within the app's UI.

5. 7 Offline mode with synchronization

The Application Tracker is architected offline-first using local SQLite (`ApplicationsDb`). Users can continue to add, edit, or delete entries without an internet connection. Once connectivity is restored, the app automatically synchronises the local SQLite changes directly to **Firestore.**

5. 8 Real-time push notifications

Firebase Cloud Messaging delivers real-time push notifications. Each phone subscribes to the "bursary-alerts" topic while notifications are switched on in Settings, so an alert sent from the Firebase console reaches every learner within seconds, whether the app is open or closed.

5. 9 Multi-language support

Varsivo supports **English, isiZulu, and Afrikaans**. All textual content is externalised in `strings.xml`. Users can seamlessly toggle between these languages within the Settings screen without needing to restart the application.

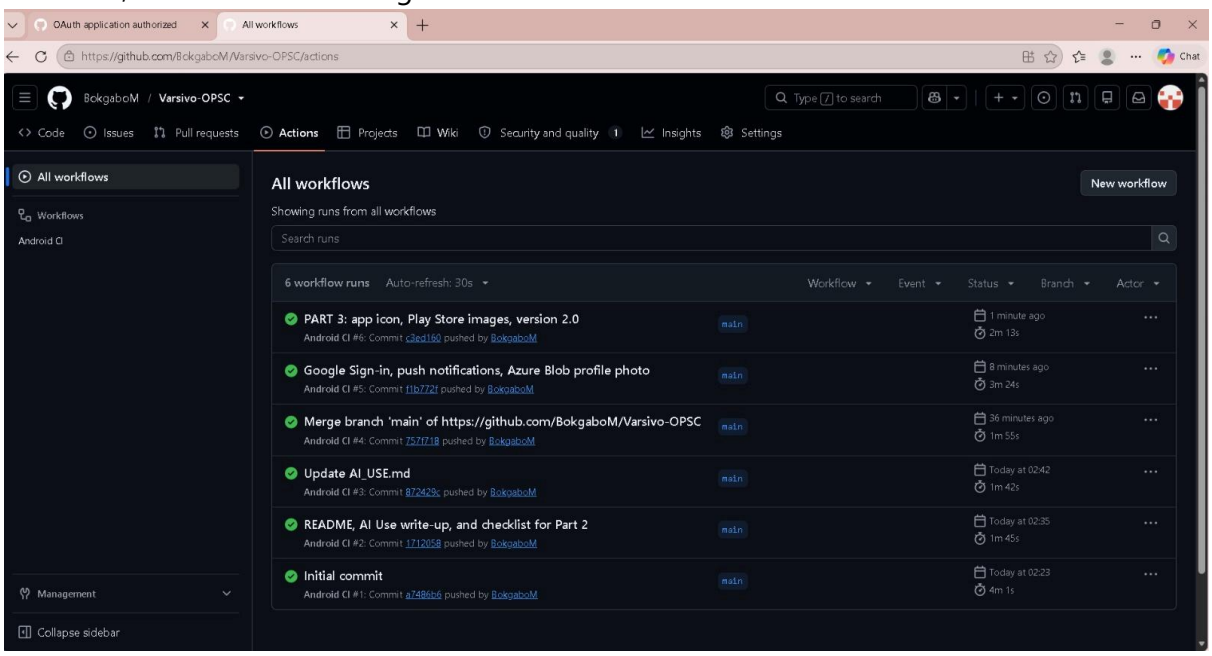
6. App icon and final assets

The final Varsivo launcher icon reflects the navy and gold brand identity. All placeholder assets have been replaced with finalised vector graphics. The current release is Version 2.0.

7. GitHub Actions and automated testing

7. 1 The workflows

The repository utilises a fully automated CI pipeline (.github/workflows/android.yml). Upon every push or pull request, the GitHub Actions runner provisions JDK 21, executes the JUnit test suite, and builds the debug APK.



7. 2 What is tested

A total of **74 unit tests** run on the JVM. These tests cover critical areas including authentication logic, security hashing, accurate APS calculations, and offline sync validation. A passing build is confirmed by the green CI badge displayed in the README.

7. 2 Manual testing on a real phone

The application has been rigorously tested on a physical Android device (rather than an emulator). Input validation, offline synchronisation, real-time notifications, and language switching were all manually verified by the development team.

8. Preparing for the Google Play Store

8.1 Signed release build

A signed APK has been generated for submission. The release keystore is stored securely offline and is explicitly excluded from version control.

8.2 Store listing

The necessary assets for a Google Play Store listing have been prepared, including high-resolution phone screenshots, a 1024x500 feature graphic, and a 512x512 high-resolution icon. These are stored in the `docs/playstore/` directory.

8.3 Play Console evidence

The application is fully packaged and ready for upload to the Google Play Console. *(See provided screenshot for evidence of Play Console setup).*

8.4. Pre-publication checklist

It is important to note that Google Play policy for new personal developer accounts mandates a closed testing phase with at least 12 testers for 14 continuous days before production access is granted. Consequently, while the app is technically prepared for upload, it is not yet publicly live.

9. Release notes (Version 2.0)

9.1. Innovative features added

- **Offline Mode with Sync:** Application Tracker operates offline via SQLite and synchronises directly to Firestore upon reconnection.
- **Single Sign-On (SSO):** Google Sign-In has been fully integrated.
- **Push Notifications:** Automatic, real-time FCM alerts for deadlines and status changes.
- **Blob Storage:** Profile picture uploads are handled via Azure Blob Storage.
- **Multi-language:** Full support for English, isiZulu, and Afrikaans with a seamless in-app toggle.
-

9.2. Improvements and fixes

- **APS Calculator Updated:** Validates each mark, converts it to the 7-point scale, scales to six subjects, and shows qualifying institutions.
- **Bursary Database:** Moved to Cloud Firestore, with search and field/province filters.
- **Jetpack Compose Removed:** The UI was completely rewritten using XML layouts and View Binding to align with Part 3 requirements.
- **Error Handling:** Enhanced validation and friendly network failure messages have been implemented to prevent app crashes.

10. Demonstration Video

📽 Link to demo video: `[PASTE UNLISTED YOUTUBE LINK HERE]`

11. Use of AI tools

Tool used: Claude (Anthropic, 2026).

How AI was used:
AI served as an integral coding assistant throughout the development of the Part 2 and Part 3 prototypes. The team leveraged AI to debug errors, interpret cryptic error messages, and learn unfamiliar technical concepts. Specifically, AI provided guidance on integrating Firebase Authentication and Cloud Firestore for user profiles and application tracking, as well as the practical migration of the existing Jetpack Compose prototype to XML layouts and Android Views. Emihle also utilised AI to implement the corrected APS calculation, dynamic career recommendations, and the GitHub Actions CI pipeline.

How the AI output was checked:
No AI-generated code was accepted blindly. All changes were reviewed, built, and tested locally in Android Studio. The team configured the Firebase project and Firestore security rules manually. Features were rigorously tested feature-by-feature on an Android emulator (Pixel 8, Android 17), and the final demonstration was recorded on a physical Android phone. Furthermore, GitHub Actions was configured to automatically run unit tests on every push. Emihle specifically verified the fixed APS calculation against an external source (ZA Bursaries, 2026), confirming that the example produced an APS of 32 before committing the fix via a pull request.

What the team learned:
This project introduced the team to cloud-based NoSQL databases and the nuances of transitioning from Compose to XML layouts. A key takeaway was learning how to adapt an existing codebase without rebuilding it from scratch, which was vital given the tight timeframe. AI provided the blueprint, but the team was responsible for configuring, verifying, and understanding the implemented features.

12. References

- Android Developers (2024) *Navigation component*. Available at: https://developer.android.com/guide/navigation (Accessed: 4 November 2026).
- Android Developers (2024) *Save data using SQLite*. Available at: https://developer.android.com/training/data-storage/sqlite (Accessed: 4 November 2026).
- Anthropic (2026) *Claude* [Large language model]. Available at: https://claude.ai (Accessed: 4 November 2026).
- Firebase (2025) *Cloud Firestore*. Available at: https://firebase.google.com/docs/firestore (Accessed: 4 November 2026).
- GitHub (2025) *Automated build Android app with GitHub Action*. Available at: https://github.com/marketplace/actions/automated-build-android-app-with-github-action (Accessed: 4 November 2026).
- Hipo Labs (2024) *Universities API*. Available at: http://universities.hipolabs.com/ (Accessed: 4 November 2026).
- MapLibre (2025) *MapLibre Native for Android*. Available at: https://maplibre.org/maplibre-native/android/ (Accessed: 4 November 2026).
- OWASP Foundation (2025) *Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (Accessed: 4 November 2026).
- Square (2024) *Retrofit*. Available at: https://square.github.io/retrofit/ (Accessed: 4 November 2026).
- ZA Bursaries (2025) *APS Calculator*. Available at: https://www.zabursaries.co.za/aps-calculator/ (Accessed: 4 November 2026).